How to write setter & getter for non static fields :

Setter Method :
----
* It is used to modify the existing object data.
* At a time we can modify only one object data.
* The setter method return type must be **void** and It must accept only one parameter.
* With the help of this parameter we can modify the existing object data.

Getter Method:
----
* It is used to read the private non static field value outside of the class.
* At a time we can read only one private data value.
* Getter method return type must be **non-void,** actually the return type will depend upon field data type.
* It is useful to read private data value outside of BLC class.

Example :

```
class Student
{
  private int marks;

  //Parameterized constructor to initialize the non static field

  public void setMarks(int marks)  //setter
  {
   this.marks = marks;
  }

  public int getMarks() //getter
  {
   return this.marks;
  }
}
```

****Encapsulation**

[Accessing the **private field** data with public methods **like** setter & getter to perform **read & write** operation]

setter (write) / **getter** (read)

Binding the private **field data** with its associated **method** in a single unit is called Encapsulation.

Encapsulation ensures that our private data (Object Properties) must be accessible via public methods like setter and getter.

It provides security because our data is private (Data Hiding) and it is only accessible via public methods WITH PROPER DATA VALIDATION.

In java, class is the example of encapsulation. We can also achieve encapsulation for static field but it is rarely used because it is common for all the objects.

How to achieve encapsulation in a class:

In order to achieve encapsulation, we should follow the following two techniques:

1) Declare all the data members with private access modifiers (Data Hiding OR Data Security)

2) Writing public methods to perform read(getter) and write(setter) operation on these private data like setter and getter.

Note: If we declare all our data with private access modifier then it is called TIGHTLY ENCAPSULATED CLASS. On the other hand, if we declare our data other than private access modifier then it is called Loosely Encapsulated class.

//Program on encapsulation :

```java
package com.ravi.encapsulation;

public class EncapsulationDemo
{
    public static void main(String[] args)
    {
      int id = Integer.parseInt(IO.readln("Enter Employee id : "));
      String name = IO.readln("Enter Employee Name :");
      double salary = Double.parseDouble(IO.readln("Enter Employee Salary :"));

      Employee scott = new Employee(id, name, salary);
      IO.println(scott);

      double increment = Double.parseDouble(IO.readln("Enter your increment amount :"));
      scott.setSalary(scott.getSalary() + increment);
      IO.println(scott);


    }

}

class Employee
{
   private int id;
   private String name;
   private double salary;

   public Employee(int id, String name, double salary)
   {
     //Data Validation
    this.id = id;
    this.name = name;
    this.salary = salary;
   }

   @Override
   public String toString()
   {
    return "Employee [id=" + id + ", name=" + name + ", salary=" + salary + "]";
   }

   public int getId()
   {
    return this.id;
   }

   public void setId(int id)
   {
    this.id = id;
   }

   public String getName()
   {
    return this.name;
   }

   public void setName(String name)
   {
    this.name = name;
   }

   public double getSalary()
   {
    return this.salary;
   }

   public void setSalary(double incrementedSalaryAmount)
   {
     if(incrementedSalaryAmount <= this.salary)
     {
         System.err.println("Increment amount cannot be zero OR -Ve");
         System.exit(0);
     }
    this.salary = incrementedSalaryAmount;
   }

}
```

Up to here How many ways we can initialize the non static fields :
----
1) By using refernce variable [Example raj.roll = 101]

2) By using Method Parameter
```
   public void setEmployeeData(int id, String name)
   {
      this.id = id;
      this.name = name;
   }
```

3) By using Parameterized Constructor
```
   public Employee(int id, String name)
   {
         this.id = id;
         this.name = name;
   }
```

4) Initialization at the time of declaration :
    double balance = 10000;

FINAL CONCLUSION:

Parameterized Constructor : To initialize the Object properties (non static Field) with user values.

Setter : To modify the existing object data. [Only one data at a time] OR Writing Operation

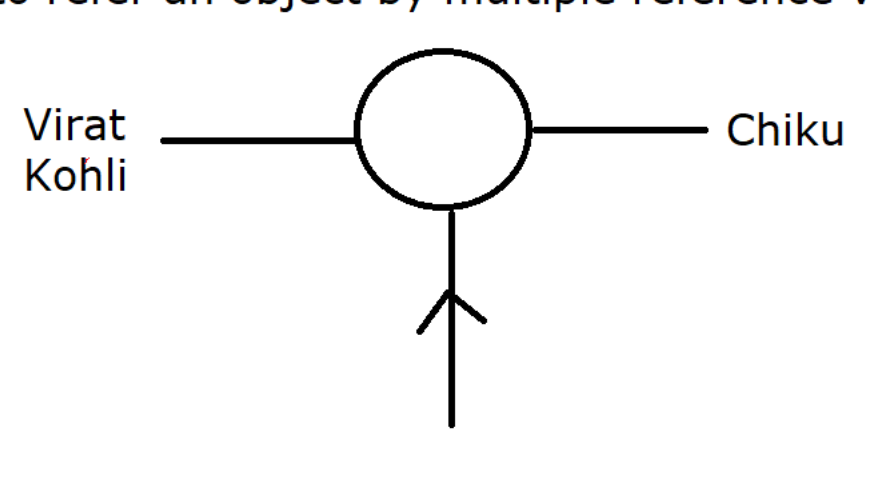
Getter : To read/retrieve private data value outside of BLC class. [Reading Operation]

toString() : To print Object properties (Non static Field)

What is Shallow and Deep copy in java:

Q1) Can we refer an object by multiple reference variable ?

Ans : Yes, It is possible to refer an object by multiple reference variables.



Q2) Can a reference variable refer to multiple objects ?
    No, At a time a reference variable can refer only one object.

Shallow copy :
----
* In Shallow copy, Only one object will be created and the same object would be referred by multiple reference variables so if we modify the object data by any of the reference variable then original object will be modified.

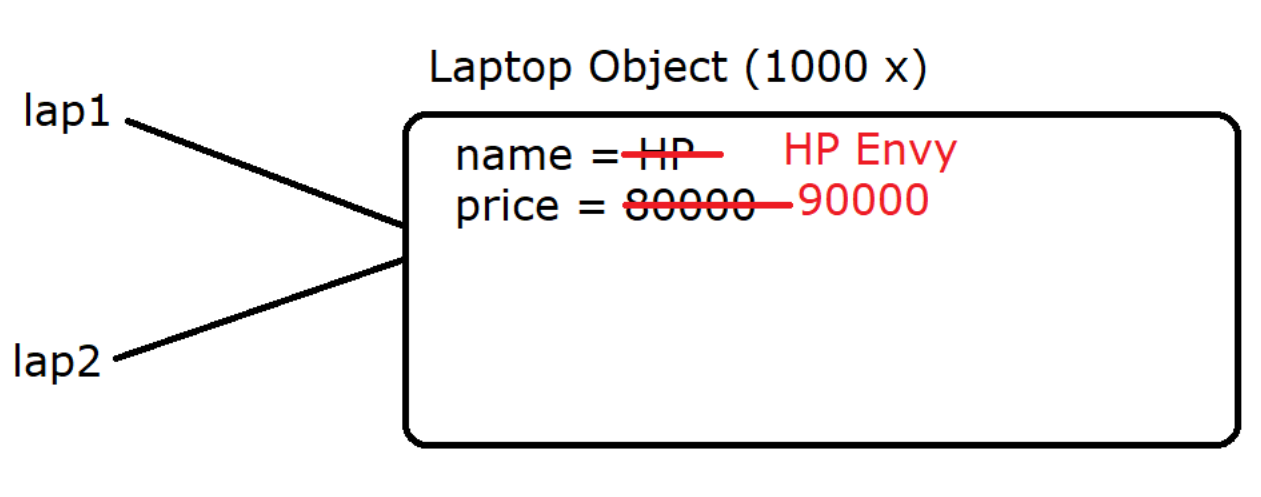
Diagram Example:

```
public static void main(String[] args)
{
 Laptop lap1 = new Laptop("HP", 80000);
 IO.println("Lap1 Data "+lap1);

 Laptop lap2 = lap1;



}
```




```java
package com.ravi.shallow_copy;

public class LaptopDemo
{
    public static void main(String[] args)
    {
        Laptop lap1 = new Laptop("HP", 80000);
        IO.println("Lap1 Data "+lap1);

        Laptop lap2 = lap1;
        lap2.setName("HP Envy");
        lap2.setPrice(90000);
        IO.println("Lap1 Data "+lap1); //HP Envy   90000
        IO.println("Lap2 Data "+lap2); //HP Envy   90000
    }
}

class Laptop
{
    private String name;
    private double price;

    public Laptop(String name, double price)
    {
        super();
        this.name = name;
        this.price = price;
    }

    public String getName()
    {
        return name;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    public double getPrice()
    {
        return price;
    }

    public void setPrice(double price)
    {
        this.price = price;
    }

    @Override
    public String toString()
    {
        return "Laptop [name=" + name + ", price=" + price + "]";
    }

}
```